

ZYROO INTERNSHIP PROGRAM

Week 2 — Vulnerability Scanning

Vulnerability Assessment Findings Report

Prepared by

Syed Mujtaba Hussain

Cybersecurity Intern

Target: Metasploitable2 (192.168.43.134) | Tool: Nikto v2.5.0 | Isolated Lab Environment

1. Executive Summary

As part of Week 2 of the ZYROO Cybersecurity Internship, a vulnerability scan was performed against a deliberately vulnerable lab target (Metasploitable2) to practice the core workflow used in real-world security assessments: scanning, analysis, and reporting. The scan uncovered multiple weaknesses ranging from exposed credentials to outdated, unsupported software. The most serious finding is a publicly accessible configuration file that reveals database credentials — in a production environment this would represent an immediate, high-priority risk requiring urgent remediation. This report summarizes the scope, methodology, key findings, and recommended fixes in a way intended to be understandable to both technical and non-technical readers.

2. Scope

The assessment was limited to a single, isolated lab target running on a private virtual network with no exposure to the public internet or any production system.

- Target: Metasploitable2 virtual machine (IP: 192.168.43.134)
- Service tested: Web server on port 80 (Apache/PHP)
- Attacker machine: Kali Linux (VMware), isolated on the same private lab network
- Testing window: Single scan session, conducted with explicit authorization within a self-owned lab environment

3. Methodology

A black-box web vulnerability scan was performed using Nikto, an open-source web server scanner that checks for outdated software, dangerous files, misconfigurations, and known vulnerability signatures.

- Command used: `nikto -h http://192.168.43.134 -output scan_result.txt`
- The scan enumerated the target's HTTP responses, headers, and accessible files/directories
- Raw output was saved to `scan_result.txt` and reviewed manually to identify and prioritize distinct vulnerabilities
- Findings were ranked by potential impact and ease of exploitation

4. Key Vulnerabilities Found

F-01 Exposed configuration file (`wp-config.php`) revealing credentials

Severity: **Critical** **Affected Service:** Port 80 / HTTP **Reference:** CWE-200 (Info Exposure)

A configuration file was found accessible via a direct HTTP request and its content indicates it contains database credentials. If this were a real WordPress-backed site, this file directly exposes database username/password, giving an attacker a path to full data compromise.

Recommended fix: Immediately restrict or remove public access to configuration files via web server rules (e.g., deny access to `*.php` config files outside the application root), rotate any exposed credentials, and ensure config files are never served as static, readable content.

F-02 phpMyAdmin database administration panel publicly accessible

Severity: **High** **Affected Service:** Port 80 / `/phpMyAdmin/` **Reference:** OWASP A05 - Security Misconfiguration

The phpMyAdmin interface, used to manage the MySQL database, is reachable without any IP restriction. An exposed admin panel is a prime target for brute-force or credential-stuffing attacks against the database layer.

Recommended fix: Restrict phpMyAdmin access to trusted/internal IP addresses only, place it behind authentication at the web-server level (e.g., HTTP basic auth) in addition to its own login, or disable it entirely if not required.

F-03 Outdated, end-of-life Apache web server (2.2.8)

Severity: **High** **Affected Service:** Port 80 / HTTP **Reference:** Apache 2.2.x branch EOL

The target runs Apache 2.2.8, which reached end-of-life at version 2.2.34 and no longer receives security patches. Outdated server software is a common entry point for attackers exploiting known, unpatched vulnerabilities.

Recommended fix: Upgrade Apache to a currently supported version (2.4.x or later) and establish a regular patch/update cycle.

F-04 phpinfo.php script exposing full server configuration

Severity: **Medium** Affected Service: Port 80 / /phpinfo.php Reference: CWE-552

A publicly reachable phpinfo() script reveals detailed information about the server's PHP configuration, installed modules, and internal file paths — information that can help an attacker plan further attacks.

Recommended fix: Remove or restrict access to phpinfo.php and any other diagnostic scripts before deployment to a live environment.

F-05 HTTP TRACE method enabled (Cross-Site Tracing risk)

Severity: **Medium** Affected Service: Port 80 / HTTP Reference: OWASP - Cross-Site Tracing (XST)

The web server accepts the HTTP TRACE method, which can be combined with cross-site scripting vulnerabilities to steal cookies or authentication headers that are normally protected from script access.

Recommended fix: Disable the TRACE (and TRACK) HTTP methods at the web server configuration level, as they are rarely needed for normal operation.

5. Risk Ranking

Findings are ranked from highest to lowest priority based on potential impact and ease of exploitation.

ID	Vulnerability	Severity	Priority Rank
F-01	Exposed configuration file (wp-config.php) revealing credentials	Critical	1
F-02	phpMyAdmin database administration panel publicly accessible	High	2
F-03	Outdated, end-of-life Apache web server (2.2.8)	High	3
F-04	phpinfo.php script exposing full server configuration	Medium	4
F-05	HTTP TRACE method enabled (Cross-Site Tracing risk)	Medium	5

6. Remediation Recommendations Summary

- Immediately secure or remove any exposed configuration files containing credentials (highest priority)
- Restrict access to administrative interfaces such as phpMyAdmin to trusted networks only
- Upgrade the web server to a currently supported, patched version
- Remove diagnostic/debug scripts (e.g., phpinfo.php) from any publicly reachable environment
- Disable unnecessary HTTP methods such as TRACE
- Add standard security headers (X-Frame-Options, X-Content-Type-Options) to reduce clickjacking and MIME-sniffing risk

7. Conclusion

This assessment successfully identified multiple vulnerabilities in the lab target, ranging from a critical credential-exposure issue to lower-severity configuration weaknesses. The exercise demonstrates the standard vulnerability assessment workflow:

scanning a target, analyzing and prioritizing results, and communicating findings in a way that supports informed remediation decisions. All testing was conducted exclusively within an isolated, self-owned lab environment in accordance with authorized testing practices.